MATLAB in Neuroscience for Non-Programmers

Homework — Session 01 · Variables, Matrices, Indexing and Operators

Your Name: __Hamid Reza Ghahramani____ **Date:** ___30 Sep 2026__

Instructions

Please complete all parts. For each task:

- Write and run the MATLAB code that solves the problem.
- Take a screenshot of your MATLAB **Command Window** (not the Workspace or Editor).
- Paste the screenshot into the answer box under that part.
- When you have finished, save the file and send it back to us.
- If something is unclear, ask in the group. Please **do not use AI to produce the final answers.** Using it to understand something better is fine, and encouraged.

> Everything here uses only what we did in class. There is no new command in this assignment.

Part A — Making your first variables

Imagine a machine has just recorded brain activity from one person. Before looking at any data, you create the variables that describe the recording.

1. Create a single number called `fs` with the value `250`. It means the machine took 250 measurements every second. Then check its size with `size(fs)`.
2. Create a text variable called `subject_id` with the value `'S01'`. Check what type it is with `class(subject_id)`.
3. Create a true/false variable called `is_clean` with the value `true`. Check its type with `class(is_clean)`.
4. Create a **row** vector called `voltages` with the values `[12, 15, 11, 9, 14]`. These are five measurements from one electrode. Check its size.
5. Create a **column** vector called `channels` with the numbers `1` to `5`, one per row. (Semicolons make new rows.)
6. Create an 8×8 table full of zeros called `empty_table`. It is an empty container you would fill in later.
7. Create an 8×8 table using `eye(8)`. Look at it, and write one sentence describing what is special about it.

   The matrix has ones on the main diagonal and zeros everywhere else.
8. Create a 4×10 table of random numbers using `randn` and call it `fake_signal`. You have just made up a recording of 4 electrodes with 10 measurements each.
9. Run `whos` and take a screenshot showing all the variables you have made.

Your Answer:

```
 >> fs=250

fs =

  250

>> size(fs)

ans =

    1    1

>> subject_id='S01'

subject_id =

   'S01'

>> class(subject_id)

ans =

   'char'

>> is_clean=true

is_clean =

  logical

  1

>> classis

Unrecognized function or variable 'classis'.

>> class(is_clean)

ans =

   'logical'

>> voltages=[14 9 11 15 12]

voltages =

   14    9   11   15   12

>> size(voltages)

ans =

    1    5

>> channels=[1;2;3;4;5]

channels =

    1
    2
```

```
   3
   4
   5

>> size(channels)

ans =

   5   1

>> empty_table=zeros(8 8)
 empty_table=zeros(8 8)
               ↑
Invalid expression. Check for missing multiplication operator, missing or unbalanced delimiters, or
other syntax error. To construct matrices, use brackets instead of parentheses.

>> empty_table=zeros(8,8)

empty_table =

   0   0   0   0   0   0   0   0
   0   0   0   0   0   0   0   0
   0   0   0   0   0   0   0   0
   0   0   0   0   0   0   0   0
   0   0   0   0   0   0   0   0
   0   0   0   0   0   0   0   0
   0   0   0   0   0   0   0   0
   0   0   0   0   0   0   0   0

>> size(empty_table)

ans =

   8   8

>> eye_table=eye(8)

eye_table =

   1   0   0   0   0   0   0   0
   0   1   0   0   0   0   0   0
   0   0   1   0   0   0   0   0
   0   0   0   1   0   0   0   0
   0   0   0   0   1   0   0   0
   0   0   0   0   0   1   0   0
   0   0   0   0   0   0   1   0
   0   0   0   0   0   0   0   1

>> eye_tablee=eye(8,7)

eye_tablee =

   1   0   0   0   0   0   0
   0   1   0   0   0   0   0
   0   0   1   0   0   0   0
   0   0   0   1   0   0   0
   0   0   0   0   1   0   0
   0   0   0   0   0   1   0
   0   0   0   0   0   0   1
   0   0   0   0   0   0   0
```

```
>> fake_signal=randn(4,10)

fake_signal =

   0.5377   0.3188   3.5784   0.7254  -0.1241   0.6715   0.4889   0.2939  -1.0689   0.3252
   1.8339  -1.3077   2.7694  -0.0631   1.4897  -1.2075   1.0347  -0.7873  -0.8095  -0.7549
  -2.2588  -0.4336  -1.3499   0.7147   1.4090   0.7172   0.7269   0.8884  -2.9443   1.3703
   0.8622   0.3426   3.0349  -0.2050   1.4172   1.6302  -0.3034  -1.1471   1.4384  -1.7115
```

```
>> whos
  Name             Size            Bytes  Class      Attributes

  ans              1x2                16  double
  channels         5x1                40  double
  empty_table      8x8               512  double
  eye_table        8x8               512  double
  eye_tablee       8x7               448  double
  fake_signal      4x10              320  double
  fs               1x1                 8  double
  is_clean         1x1                 1  logical
  subject_id       1x3                 6  char
  voltages         1x5                40  double
```

Part B — Reading a table of measurements

Type this table into MATLAB exactly as written. Read it as **4 electrodes** (the rows) measured at **6 moments in time** (the columns).

```
eeg = [11 12 13 14 15 16;
       21 22 23 24 25 26;
       31 32 33 34 35 36;
       41 42 43 44 45 46];
```

> The numbers were chosen on purpose: the first digit is the **row** and the second digit is the **column**. So `eeg(3,5)` should give `35`. You can always check your own answer.

1. Run `size(eeg)`. How many rows and how many columns are there?
2. Get the whole of **row 3** — everything electrode 3 measured.
3. Get the whole of **column 5** — what all four electrodes measured at that one moment.
4. Get the single value in **row 2, column 4**.
5. Using `end`, get the value in the **last row, last column**.
6. Get the block made of **rows 1 to 2 and columns 2 to 4**.
7. Every number in this table is half of what it should be. Multiply the whole table by `2` and save the result in a new variable called `eeg_fixed`.
8. Add `100` to **row 1 only**, without changing the other rows.
9. Try `eeg(5,1)`. Write down the error message you get, and explain in one sentence why it happened.

   Index in position 1 exceeds array bounds. Index must not exceed 4.

10. Electrode 2 turned out to be broken. Delete **row 2**, then run `size(eeg)` again to confirm only 3 rows are left.

Your Answer:

```
>> egg=[11 12 13 14 15 16;21 22 23 24 25 26;31 32 33 34 35 36;41 42 43 44 45 46]

egg =

   11   12   13   14   15   16
   21   22   23   24   25   26
   31   32   33   34   35   36
   41   42   43   44   45   46

>> clear egg
>> egg
Unrecognized function or variable 'egg'.

Did you mean:
>> eeg=[11 12 13 14 15 16;21 22 23 24 25 26;31 32 33 34 35 36;41 42 43 44 45 46]

eeg =

   11   12   13   14   15   16
   21   22   23   24   25   26
   31   32   33   34   35   36
   41   42   43   44   45   46

>> size(eeg)

ans =

    4    6

>> eeg(3 :)
 eeg(3 :)
       ↑
Invalid expression. When calling a function or indexing a variable, use parentheses. Otherwise, check
for mismatched delimiters.

>> eeg(3,:)

ans =

   31   32   33   34   35   36

>> eeg(:,5)

ans =

   15
   25
   35
   45

>> eeg(2,4
 eeg(2,4
       ↑
Invalid expression. When calling a function or indexing a variable, use parentheses. Otherwise, check
for mismatched delimiters.
```

```
Did you mean:
>> eeg(2,4)

ans =

   24

>> eeg(end,end)

ans =

   46

>> eeg(1:2,2:4)

ans =

   12   13   14
   22   23   24

>> eeg_fixed=eeg*2

eeg_fixed =

   22   24   26   28   30   32
   42   44   46   48   50   52
   62   64   66   68   70   72
   82   84   86   88   90   92

>> eeg(1,:)+100

ans =

  111  112  113  114  115  116

>> eeg

eeg =

   11   12   13   14   15   16
   21   22   23   24   25   26
   31   32   33   34   35   36
   41   42   43   44   45   46

>> eeg(1,:)=eeg(1,:)+100

eeg =

  111  112  113  114  115  116
   21   22   23   24   25   26
   31   32   33   34   35   36
   41   42   43   44   45   46

>> eeg(5,1)
Index in position 1 exceeds array bounds. Index must not exceed 4.

>> eeg(2,:)=[]

eeg =
```

```
  111  112  113  114  115  116
   31   32   33   34   35   36
   41   42   43   44   45   46

>> size(eeg)

ans =

    3    6
```

```
>> whos
  Name              Size              Bytes  Class      Attributes

  ans               1x2                  16  double
  channels          5x1                  40  double
  eeg               3x6                 144  double
  eeg_fixed         4x6                 192  double
  empty_table       8x8                 512  double
  eye_table         8x8                 512  double
  eye_tablee        8x7                 448  double
  fake_signal       4x10                320  double
  fs                1x1                   8  double
  is_clean          1x1                   1  logical
  subject_id        1x3                   6  char
  voltages          1x5                  40  double
```

Part C

```
a = [1 2; 3 4];
b = [5 6; 7 8];
```

1. Add the two tables together: `a + b`. Then subtract: `a - b`.
2. Compute `a .* b` — **with** the dot. Write down the four numbers you get.
3. Compute `a * b` — **without** the dot. Write down the four numbers you get.
4. The two answers are not the same. In one sentence, say what the dot changed.

   The dot makes MATLAB multiply corresponding elements in the same positions instead of performing matrix multiplication.
5. Try `a + [1 2 3]`. Write down the error message, and say in one sentence why it happened.

   The error happened because a is a 2×2 matrix, while [1 2 3] is a 1×3 vector, so their sizes are incompatible for addition.

> **Hint.** With the dot, MATLAB pairs up the numbers sitting in the **same position**: top-left with top-left, top-right with top-right. Without the dot it uses the rule from mathematics, where a whole **row** of `a` meets a whole **column** of `b` and the results are added together.

Your Answer:

```
>> a=[1 2 ;3 4]

a =

   1   2
   3   4

>> b=[5 6 ;7 8]

b =

   5   6
   7   8

>> a+b

ans =

   6   8
  10   12

>> a-b

ans =

  -4  -4
  -4  -4

>> a.*b

ans =

   5  12
  21  32

>> a*b

ans =

  19  22
  43  50

>> a+[1 2 3]
Arrays have incompatible sizes for this operation.
```

Related documentation

Part D —short tasks

1. Create `eeg_samples = [-50, -30, 0, 20, 60, 100]`. Make a list of true/false values showing where the number is greater than `0`, then use `sum` to count how many passed.
2. You repeated a test 10 times. The odd-numbered runs showed a picture, the even-numbered runs played a sound. Create `trials = 1:10`, then use `mod` to make two vectors: `picture_trials` and `sound_trials`.

Your Answer:

```
>> egg_samples=[-50 -30 0 20 60 100]

egg_samples =

  -50  -30    0   20   60  100

>> eeg_samples>0
Unrecognized function or variable 'eeg_samples'.

Did you mean:
>> egg_samples>0

ans =

  1×6 logical array

  0  0  0  1  1  1

>> sum(egg_samples>0)

ans =

    3

>> trials=1:10

trials =

    1    2    3    4    5    6    7    8    9   10

>> mod(trials,2)

ans =

    1    0    1    0    1    0    1    0    1    0

>> picture_trials=trials(mod(trials,2)>0)

picture_trials =

    1    3    5    7    9

>> sound_trials=trials(mod(trials,2)==0)

sound_trials =

    2    4    6    8   10

>> length(picture_trials)

ans =

    5

>> lenght(sound_trials)
Unrecognized function or variable 'lenght'.

Did you mean:
```

```
>> length(sound_trials)

ans =

   5

>> egg_samples>0&egg_samples<100

ans =

 1×6 logical array

  0  0  0  1  1  0

>> egg_samples(egg_samples>0&egg_samples<100)

ans =

   20   60

>> sum(0&egg_samples<100)

ans =

   0

>> sum(eeg_samples > 0 & eeg_samples < 100)
Unrecognized function or variable 'eeg_samples'.

Did you mean:
>> sum(egg_samples > 0 & egg_samples < 100)

ans =

   2
```

Part E — Challenge

This one needs a little thinking, but it uses nothing new. Take your time. If you get stuck, write down where you got stuck — that counts too.

Four electrodes were recorded at five moments in time. Electrode 2 came loose during the recording, so two of its values are far too large:

```
signal = [ 5   7   6   8   5;
           4 200   3   5 210;
           6   5   7   6   5;
           3   4   5   4   6];
```

1. Check the size of `signal`. How many electrodes, and how many moments in time?

   There are **4 electrodes** and **5 moments in time**.

2. Make a list of true/false values showing which measurements in **row 2** are greater than `100`.

3. Use `sum` on that list to count how many bad values row 2 has.
4. Replace only those bad values with `0`, leaving the good values in row 2 untouched. Print row 2 to check.
5. Now delete row 2 completely, because the electrode was broken. Check the new size.
6. **The thinking part:** before you deleted it, electrode 4 was in row 4. After deleting row 2, which row holds electrode 4? Write your answer, then prove it by printing that row.

   Electrode 4 is now in **Row 3** because Row 2 was deleted, so the rows below it moved up by one.

> **Hint for the last task:** when a row is removed, every row below it moves up by one. You can check whether you are right by comparing the numbers you print with the original table above.

Your Answer:

```
>> signal=[5 7 6 8 5;4 200 3 5 210;6 5 7 6 5;3 4 5 4 6]

signal =

   5   7   6   8   5
   4 200   3   5 210
   6   5   7   6   5
   3   4   5   4   6

>> size(signal)

ans =

   4   5

>> signal(2,:)>100

ans =

  1×5 logical array

  0  1  0  0  1

>> sum(signal(2,0)>100)
Index in position 2 is invalid. Array indices must be positive integers or logical values.

>> sum(signal(2,:)>100)

ans =

   2

>> signal(2,signal(2,:)>100
 signal(2,signal(2,:)>100
                ↑
Invalid expression. When calling a function or indexing a variable, use parentheses. Otherwise, check
for mismatched delimiters.

Did you mean:
>> signal(2,signal(2,:)>100)

ans =
```

```
  200   210

>> signal(2,signal(2,:)>100)=0

signal =

    5    7    6    8    5
    4    0    3    5    0
    6    5    7    6    5
    3    4    5    4    6

>> signal(2,:)

ans =

    4    0    3    5    0

>> signal(2,:)[]
 signal(2,:)[]
         ↑
Invalid expression. When calling a function or indexing a variable, use parentheses. Otherwise, check
for mismatched delimiters.

>> signal(2,:)[]
 signal(2,:)[]
         ↑
Invalid expression. When calling a function or indexing a variable, use parentheses. Otherwise, check
for mismatched delimiters.

>> signal(2,:)=[]

signal =

    5    7    6    8    5
    6    5    7    6    5
    3    4    5    4    6

>> size(signal)

ans =

    3    5

>> signal(3,:)

ans =

    3    4    5    4    6

>>
```

Every line of code you write today is one more thing the brain can no longer hide from you.